

练习

对实验报告的要求：

- 基于markdown格式来完成，以文本方式为主
- 填写各个基本练习中要求完成的报告内容
- 完成实验后，请分析ucore_lab中提供的参考答案，请在实验报告中说明你的实现与参考答案的区别
- 列出你认为本实验中重要的知识点，以及与对应的OS原理中的知识点，并简要说明你对二者的含义，关系，差异等方面的理解（也可能出现实验中的知识点没有对应的原理知识点）
- 列出你认为OS原理中很重要，但在实验中没有对应上的知识点

练习0：填写已有实验

本实验依赖实验1/2。请把你做的实验1/2的代码填入本实验中代码中有“LAB1”,“LAB2”的注释相应部分。

练习1：理解基于FIFO的页面替换算法（思考题）

描述FIFO页面置换算法下，一个页面从被换入到被换出的过程中，会经过代码里哪些函数/宏的处理（或者说，需要调用哪些函数/宏），并用简单的一两句话描述每个函数在过程中做了什么？（为了方便同学们完成练习，所以实际上我们的项目代码和实验指导的还是略有不同，例如我们将FIFO页面置换算法头文件的大部分代码放在了kern/mm/swap_fifo.c文件中，这点请同学们注意）

- 至少正确指出10个不同的函数分别做了什么？如果少于10个将酌情给分。我们认为只要函数原型不同，就算两个不同的函数。要求指出对执行过程有实际影响,删去后会导致输出结果不同的函数（例如assert）而不是cprintf这样的函数。如果你选择的函数不能完整地体现“从换入到换出”的过程，比如10个函数都是页面换入的时候调用的，或者解释功能的时候只解释了这10个函数在页面换入时的功能，那么也会扣除一定的分数

练习2：深入理解不同分页模式的工作原理（思考题）

get_pte()函数（位于kern/mm/pmm.c）用于在页表中查找或创建页表项，从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下，是实现虚拟内存与物理内存之间映射关系非常重要的内容。

- get_pte()函数中有两段形式类似的代码，结合sv32，sv39，sv48的异同，解释这两段代码为什么如此相像。
- 目前get_pte()函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

练习3：给未被映射的地址映射上物理页（需要编程）

补充完成do_pgfault（mm/vmm.c）函数，给未被映射的地址映射上物理页。设置访问权限 的时候需要参考页面所在 VMA 的权限，同时需要注意映射物理页时需要操作内存控制 结构所指定的页表，而不是内核的页表。请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 请描述页目录项（Page Directory Entry）和页表项（Page Table Entry）中组成部分对ucore实现页替换算法的潜在用处。
- 如果ucore的缺页服务例程在执行过程中访问内存，出现了页访问异常，请问硬件要做哪些事情？

- 数据结构Page的全局变量（其实是一个数组）的每一项与页表中的页目录项和页表项有无对应关系？如果有，其对应关系是啥？

练习4：补充完成Clock页替换算法（需要编程）

通过之前的练习，相信大家对FIFO的页面替换算法有了更深入的了解，现在请在我们给出的框架上，填写代码，实现 Clock页替换算法（mm/swap_clock.c）。请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 比较Clock页替换算法和FIFO算法的不同。

练习5：阅读代码和实现手册，理解页表映射方式相关知识（思考题）

如果我们采用“一个大页”的页表映射方式，相比分级页表，有什么好处、优势，有什么坏处、风险？

扩展练习 Challenge：实现不考虑实现开销和效率的LRU页替换算法（需要编程）

challenge部分不是必做部分，不过在正确最后会酌情加分。需写出有详细的设计、分析和测试的实验报告。完成出色的可获得适当加分。

虚拟内存系统中的页面错误（Page Fault）是指发生当程序尝试访问尚未载入物理内存的虚拟地址所对应的页时产生的异常。处理页面错误的一般流程如下：

异常触发： 当程序尝试访问尚未加载到物理内存的虚拟页时，会触发页面错误异常。

异常处理： 处理器捕获到这个异常，操作系统内核将负责处理异常。当发生页面错误时，处理器暂停当前程序的执行，并传递控制到操作系统内核。

操作系统响应： 操作系统内核接管控制权，对页面错误进行处理。通常，操作系统会执行以下操作：

确定缺页类型： 操作系统检查导致页面错误的原因，可以是缺页异常（缺少页表条目）或是非法访问（例如，对只读内存区域的写操作）等。

加载缺页： 如果是缺页异常，操作系统会尝试将所需的页面加载到物理内存中。这可能涉及从磁盘上的交换空间中读取数据，或者是从其他存储设备加载页面数据。

更新页表： 操作系统会更新页表，将虚拟地址映射到新加载到物理内存的页面。

恢复执行： 当缺页处理完成后，操作系统允许被中断的程序继续执行。在操作系统加载了缺失的页面之后，处理器重新执行原来发生页面错误的指令。

恢复执行： 一旦操作系统成功加载了缺页所需的页面，处理器恢复执行中断的程序。程序继续执行缺页引起异常的指令。

在操作系统中，页替换算法是用于管理虚拟内存的一部分。当所有物理页面（内存页）都已分配且需要为新的页面腾出空间时，页替换算法就会起作用。这种情况下，操作系统需要在内存中选择一个页面作为牺牲对象，即将其替换出去，以便为新的页面提供空间。

页替换算法的目标是尽可能地提高性能并减少页面错误的数量，同时遵守特定的替换策略。以下是一些常见的页替换算法：

最佳 (Optimal) 替换算法： 选择未来最长时间不会被使用的页面进行替换。这是理论上最佳的算法，但实际上很难在实时系统中实现，因为它需要对未来的页面访问模式有精确预测。

先进先出 (FIFO) 替换算法： 将最早进入内存的页面替换出去。这种方法简单易实现，但可能会出现 "Belady's Anomaly" (即在增加内存时，页面错误数反而会增加)。

最近最久未使用 (LRU) 替换算法： 根据页面最近一次被访问的时间进行替换。LRU需要维护最近访问的记录，因此实现相对较为复杂。

时钟 (Clock) 替换算法： 将页面组织为环形链表，使用一个指针按顺序扫描页面。当需要替换时，检查指针所指页面的访问位 (是否被访问过)，如果为0则替换；如果为1，则将访问位设为0并继续扫描。

最少使用 (LFU) 替换算法： 替换最少被访问的页面。LFU需要维护页面被访问的次数，实现较为复杂。

在操作系统中，实现这些页替换算法通常需要使用数据结构 (如栈、队列、链表或散列表) 来追踪页面的使用情况，并为不同的算法选择最佳的数据结构。同时，页替换算法会影响系统的性能和页面错误率，因此选择合适的算法对系统的效率和资源利用非常重要。

多级页表是一种页表的组织结构，用于处理大型虚拟地址空间，其将整个虚拟地址空间分层管理，以便更有效地管理内存。当处理缺页异常时，多级页表需要执行以下步骤：

页表查找： 当CPU试图访问一个虚拟地址时，硬件首先根据该虚拟地址检查页表。在多级页表中，需要进行多次查找才能定位到实际的物理地址。这些页表的层次结构可以是两级、三级或更多级，每一级页表指向下一级页表，直到找到最终的物理页框。

缺页异常处理： 如果某个虚拟地址的页表项 (Page Table Entry, PTE) 没有被加载到内存中，会触发缺页异常。在这种情况下，操作系统会介入，根据缺页地址从磁盘或其他存储介质中加载相应的页面数据到物理内存中，并更新页表项，以便下一次访问时可以找到该地址对应的物理内存位置。

页面置换算法： 如果没有空闲物理页来存储新加载的页面，需要进行页面置换。通过所选的页面置换算法 (如LRU、FIFO、Clock等)，从物理内存中选择一个牺牲页，释放其占用的物理内存页框。通常，牺牲页是根据特定的置换算法来确定的。这个页面会被写回磁盘，如果它是脏页 (即已被修改)，然后操作系统将新加载的页面放置在这个物理页框中，更新页表项，使其指向新的物理内存位置。

多级页表和页面置换算法相互配合，以便处理大型虚拟地址空间和管理内存页。这种方法可以提高系统的内存管理效率，同时也增加了硬件和操作系统的复杂性。选择合适的页表级数和页面置换算法，对于系统的性能和资源利用非常重要。

本次实验是在实验二的基础上，借助于页表机制和实验一中涉及的中断异常处理机制，完成Page Fault异常处理和部分页面替换算法的实现，结合磁盘提供的缓存空间，从而能够支持虚存管理，提供一个比实际物理内存空间“更大”的虚拟内存空间给系统使用。这个实验与实际操作系统中的实现比较起来要简单，不过需要了解实验一和实验二的具体实现。实际操作系统系统中的虚拟内存管理设计与实现是相当复杂的，涉及到与进程管理系统、文件系统等的交叉访问。如果大家有余力，可以尝试完成扩展练习，实现LRU页替换算法。

本实验依赖实验2。请把你做的实验2的代码填入本实验中代码中有“LAB2”的注释相应部分。(建议手动补充，不要直接使用merge)

理解基于FIFO的页面替换算法（思考题）

FIFO（First In, First Out）是最简单的页面替换算法之一，它按照页进入物理内存的顺序进行替换。当发生缺页异常且没有空闲物理页框可供使用时，FIFO算法选择最早进入内存的页面进行替换。

其实现方式是通过一个页表（FIFO队列）来维护物理内存中的页面顺序。当页面被载入内存时，加入队尾；当发生缺页异常时，从队头取出最早进入内存的页面进行替换。

下面是一个简单的伪代码示例：

1. 初始化FIFO队列为空
2. 缺页异常处理：
 - a. 如果所需页面不在内存中（缺页）：
 - i. 如果有空闲物理页框：
 - a. 将所需页面加载到空闲物理页框中
 - b. 将该物理页框添加到FIFO队列的尾部
 - ii. 如果没有空闲物理页框：
 - a. 从FIFO队列的头部取出最早进入的页面（最先进入内存的页面）
 - b. 将该页面替换为新页面
 - c. 将新页面加载到该物理页框中
 - d. 更新页表

尽管FIFO算法实现简单，但其性能并不一定优于其他替换算法。因为FIFO算法没有考虑页面的访问频率，可能会导致“Belady异常”，即在页面数量增加时缺页率反而上升。其他替换算法，如LRU（Least Recently Used）考虑了页面的使用频率，因此通常会有更好的性能。

描述FIFO页面置换算法下，一个页面从被换入到被换出的过程中，会经过代码里哪些函数/宏的处理（或者说，需要调用哪些函数/宏），并用简单的一两句话描述每个函数在过程中做了什么？（为了方便同学们完成练习，所以实际上我们的项目代码和实验指导的还是略有不同，例如我们将FIFO页面置换算法头文件的大部分代码放在了kern/mm/swap_fifo.c文件中，这点请同学们注意）

在FIFO页面置换算法中，页面从被换入到被换出的过程会经过以下函数/宏的处理：

`_fifo_init_mm(struct mm_struct *mm)`：用于初始化FIFO算法，管理所有可换出的页面。通过`list_init`初始化FIFO队列，并将`mm->sm_priv`指向`pra_list_head`。

`_fifo_map_swappable(struct mm_struct *mm, uintptr_t addr, struct Page *page, int swap_in)`：将所需页面连接到`pra_list_head`的末尾，表示最近进入内存。通过`list_add`将页面添加到队列末尾。

`_fifo_swap_out_victim(struct mm_struct *mm, struct Page ** ptr_page, int in_tick)`：根据FIFO算法，从队列前部取出最早进入内存的页面。通过`list_prev`取得队列的最早进入的页面，然后`list_del`移除该页面，将其地址设置到`ptr_page`指针中。

`_fifo_check_swap()`：一个测试函数，模拟了多次页面置换操作，进行了多次对不同内存地址的写入操作，模拟了缺页异常的过程。

这些函数包含了FIFO页面置换算法的关键逻辑，用于管理页面的进出过程以及置换逻辑，从而实现最先进入的页面最先被置换出去的策略。

至少正确指出10个不同的函数分别做了什么？如果少于10个将酌情给分。我们认为只要函数原型不同，就算两个不同的函数。要求指出对执行过程有实际影响，删去后会导致输出结果不同的函数（例如assert）而不是cprintf这样的函数。如果你选择的函数不能完整地体现“从换入到换出”的过程，比如10个函数都是页面换入的时候调用的，或者解释功能的时候只解释了这10个函数在页面换入时的功能，那么也会扣除一定的分数

深入理解不同分页模式的工作原理（思考题）

不同的分页模式指的是不同的虚拟内存管理方式。其中最常见的是分页式和段式。

分页式（Paging）：

工作原理：将物理内存和虚拟内存分为固定大小的页面（通常为4KB或4MB），虚拟地址空间同样划分为相同大小的页。当程序需要访问内存时，CPU生成的虚拟地址被映射到物理地址。该映射是通过页表实现的，其中记录了虚拟页到物理页的映射关系。

特点：分页能够提供更大的地址空间，且页面之间是相互独立的。可以使用交换技术，允许部分页面不在物理内存中，而是存储在磁盘中。

段式（Segmentation）：

工作原理：将内存划分为逻辑上独立的段，如代码段、数据段等。每个段的长度是不固定的，它们由段表进行映射。段表中包含了每个段的起始地址和长度。

特点：提供了更好的逻辑划分，允许程序员将程序划分为不同的逻辑段。然而，它对于地址空间的管理可能并不如分页灵活。

有些系统结合使用两种模式，如x86架构中使用了分段和分页相结合的模式，称为分段分页式。这种模式同时利用了分段的逻辑划分和分页的地址翻译。

get_pte()函数（位于kern/mm/pmm.c）用于在页表中查找或创建页表项，从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下，是实现虚拟内存与物理内存之间映射关系非常重要的内容。

get_pte()函数中有两段形式类似的代码，结合sv32，sv39，sv48的异同，解释这两段代码为什么如此相像。目前get_pte()函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

在RISC-V中，不同的页表级别使用了不同数量的页表项（Page Table Entry, PTE）来对虚拟地址进行翻译。

sv32、sv39和sv48的异同：

SV32：

使用两级页表结构。

32 位虚拟地址空间，将其分为虚拟页号（VPN）和页内偏移（Page Offset）。

具有一级页表和二级页表，其中一级页表索引有 10 位，二级页表索引有 10 位。

SV39：

使用三级页表结构。

39 位虚拟地址空间，将其分为虚拟页号（VPN）和页内偏移（Page Offset）。

具有一级页表、二级页表和三级页表，其中一级页表索引有 9 位，二级页表索引有 9 位，三级页表索引有 9 位。

SV48：

使用四级页表结构。

48 位虚拟地址空间，将其分为虚拟页号（VPN）和页内偏移（Page Offset）。

具有一级页表、二级页表、三级页表和四级页表，其中一级页表索引有 9 位，二级页表索引有 9 位，三级页表索引有 9 位，四级页表索引有 9 位。

这些结构中，虚拟地址被划分为不同级别的页表索引和页内偏移。更高级别的页表存储的是更大范围的地址，这允许更多的页表级别来寻址更大的物理内存。SV39 和 SV48 是为支持更大的虚拟地址空间而引入的，以满足更高内存需求的应用场景。每个页表项所包含的字段可能略有不同，但其基本结构类似，例如 PTE_P（是否存在）、PTE_W（是否可写）、PTE_U（是否用户可访问）等字段。

get_pte() 函数中的相似段：

这两段代码在不同级的页表中起到了相似的作用，即检索对应虚拟地址的页表项。第一段代码针对的是一级页表（PDE），而第二段则是针对第二级页表（PTE）。

其相似性在于两个级别的页表查找与分配都是由相似的操作步骤完成。它们都会检查目标页表项是否存在，如果不存在则创建该页表项，并进行相应的内存分配以建立对应的映射关系。

是否将两个功能拆分：

将页表项的查找和分配分开是一种更模块化和清晰的设计，有助于提高代码的可读性和可维护性。这种拆分使得代码更易于理解和调试。

然而，现有的设计在一定程度上将两个相关的操作合并在一个函数中，这有助于简化逻辑和提高代码效率，因为避免了在两个函数之间频繁的上下文切换。同时也节省了一些函数调用的开销。因此，在特定的应用场景下，这种设计可能更为有效。

综上所述，代码中将查找和分配页表项的功能合并在一个函数中，这种设计在逻辑和性能上有其优势，但对于代码的清晰度和可维护性来说，拆分这两个功能为两个独立的函数会更有帮助。

给未被映射的地址映射上物理页（需要编程）

步骤如下：

分配一个物理页：首先需要从物理页框池中分配一个物理页。这个物理页应该是未被使用的，并且能够被映射到当前需要进行映射的虚拟地址。

构建页表项：在适当的页表中，找到对应的页表项。通常这涉及到对多级页表结构进行访问，找到页表项的位置。这些页表项描述了虚拟地址到物理地址的映射关系。

建立映射关系：修改所找到的页表项，将其内容设置为指向刚分配的物理页的物理地址。这一步是建立虚拟地址和物理地址之间的映射关系。

清除 TLB (Translation Lookaside Buffer) 中的缓存：如果之前的虚拟地址和物理地址的映射关系已经被缓存在 TLB 中，需要将 TLB 中的对应条目清除，以确保新的映射关系可以立即生效。

更新页面属性：在建立完映射关系后，可能需要设置对应页表项的一些属性，例如标志位（例如读写标志、存在标志等）。

补充完成 do_pgfault (mm/vmm.c) 函数，给未被映射的地址映射上物理页。设置访问权限的时候需要参考页面所在 VMA 的权限，同时需要注意映射物理页时需要操作内存控制结构所指定的页表，而不是内核的页表。

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

请描述页目录项（Page Directory Entry）和页表项（Page Table Entry）中组成部分对ucore实现页替换算法的潜在用处。

在 UCore 中，页目录项（Page Directory Entry，PDE）和页表项（Page Table Entry，PTE）的各个组成部分对实现页替换算法具有潜在用途：页目录项（PDE）的组成部分：

Present/Absent (P) Bit：此位表示该页目录项所指向的页表是否存在于内存中。在页替换算法中，若页面不在内存中，需要进行页面置换，将磁盘中的页面数据换入内存。

Accessed (A) Bit：此位表示该页目录项所指向的页表或页是否被访问。在许多页面置换算法中，通过检查页面访问情况来确定哪些页面最适合被置换。

Dirty (D) Bit：此位表示该页目录项所指向的页表或页是否被写入过。当页面被写入后，内存中的页面与磁盘中的数据不同步。在页替换算法中，为了保持数据一致性，可能需要将脏页面写回磁盘。

页表项（PTE）的组成部分：

Present/Absent (P) Bit：类似于页目录项，此位表示页面是否存在于内存中。如果页面不在内存中，可能需要将其从磁盘交换回内存。

Accessed (A) Bit：记录页面是否被访问，可用于辅助页替换算法，选择替换被访问较少的页面。

Dirty (D) Bit：记录页面是否被写入，如果页面被修改，可能需要将其写回磁盘。

这些位可以用于页置换算法，以确定哪些页面应该被置换，或者在页面置换过程中需要执行何种操作。例如，使用页面访问位和脏位，可以实现简单的最近最少使用（LRU）或最少使用（LFU）算法，以选择最佳页面置换策略。

如果ucore的缺页服务例程在执行过程中访问内存，出现了页访问异常，请问硬件要做哪些事情？

当 ucore 的缺页服务例程在执行过程中出现页访问异常，硬件会执行以下步骤：

触发异常处理：当出现页访问异常时，硬件将立即暂停正在执行的指令，并触发异常处理过程。

转移控制权：硬件保存当前执行指令的状态并将控制权转移到异常处理程序，该程序通常位于操作系统内核中。

异常处理程序：异常处理程序会根据异常类型（这里是页访问异常）采取相应的处理步骤。对于缺页异常，异常处理程序会尝试解决缺页，通常包括：

寻找页面：尝试找到引起缺页的页面，通常需要从磁盘或其他存储介质中将页面调入内存。

更新页表：将找到的页面映射到合适的虚拟地址，通常需要更新页表来确保合适的页面被映射到合适的地址空间。

恢复状态：在完成缺页处理后，异常处理程序会重新启动或恢复之前被中断的指令。这可能包括

将异常指令重执行或转移到下一条指令。

处理完成：当异常处理程序完成所需的页面处理操作后，控制权将返回到引发异常的指令，程序继续执行。

总体来说，硬件会负责中断当前指令的执行，传递控制权给操作系统内核中的异常处理程序，由操作系统负责识别和解决缺页异常，最终允许程序继续执行。

数据结构Page的全局变量（其实是一个数组）的每一项与页表中的页目录项和页表项有无对应关系？如果有，其对应关系是啥？

在操作系统中，数据结构 Page 中的全局变量通常是用于管理实际的物理页面。每个 Page 对象表示系统中的一个物理页面。这些物理页面会被分配给进程的虚拟地址空间，并在页表中建立虚拟地址到物理地址的映射。

通常情况下，操作系统的页表中的页目录项和页表项与 Page 数据结构中的物理页面有着直接的对应关系。具体来说：

页目录项 (Page Directory Entry) 是页表的顶层结构，用于指向页表。
页表项 (Page Table Entry) 存储着虚拟地址和物理地址之间的映射关系。

这些页表项 (Page Table Entry) 会引用 Page 结构中的对应物理页面。每个页表项都指向物理内存中的某一页，该页面对应于一个 Page 结构。

总体而言，Page 数据结构中的每个元素代表着物理内存中的一个页面，而页表项则将这些物理页面映射到进程的虚拟地址空间中。页面表项中的内容（例如物理地址）与 Page 数据结构中相应页面的信息是相关联的。

补充完成Clock页替换算法（需要编程）通过之前的练习，相信大家对FIFO的页面替换算法有了更深入的了解，现在请在我们给出的框架上，填写代码，实现 Clock页替换算法（mm/swap_clock.c）。(提示:要输出 curr_ptr的值才能通过make grade)

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

比较Clock页替换算法和FIFO算法的不同。

Clock和FIFO都是页面置换算法，它们都用于管理页面缓存中的页，但它们有一些关键区别：

工作原理：

FIFO (First In, First Out)：按照最早进入页面缓存的页面顺序进行替换。类似排队原则，最先进入的页面最先被替换出去。

Clock：也被称为“二次机会”算法。Clock是基于FIFO的基础上进行改进，它使用了类似环形缓冲区的方式组织页面，有一个指针顺序扫描页面。当需要替换页面时，Clock算法会跟踪这个指针，检查被扫描到的页面的访问位。如果访问位为0，表明页面未被访问，此时这个页面可以被替换。如果访问位为1，说明页面被访问过，那么会将访问位置0，继续扫描。

替换策略：

FIFO：FIFO在页面置换时，总是选择最早进入的页面进行替换，不考虑页面的访问频率或重要性。

Clock：Clock算法会尽量选择那些没有被访问或访问次数较少的页面进行替换。这种方法使得访问频率高的页面有机会留在缓存中，因为它们的访问位会被周期性地重置，以提供“第二次机会”。

性能：

FIFO：实现简单，但可能存在Belady异常，即页面数增加时缺页率反而增加的问题。

Clock：Clock算法相对于FIFO算法来说，更具有性能优势。由于Clock算法考虑了页面的访问情况，因此有助于提高缓存命中率，减少不必要的页面置换。

总的来说，Clock算法是FIFO算法的改进版本，增加了对页面访问情况的考量，更加智能地进行页面替换，因此在一定程度上提高了性能。

阅读代码和实现手册，理解页表映射方式相关知识（思考题）如果我们采用“一个大页”的页表映射方式，相比分级页表，有什么好处、优势，有什么坏处、风险？

“一个大页”指的是大页面的页表映射方式。相比于分级页表，这种方式具有一些优势和劣势。

优势：

更高效的TLB缓存利用：大页面可以减少页表项的数量，因此在TLB（Translation Lookaside Buffer）中可以缓存更多的页表项，提高TLB的命中率。这能够减少CPU访问内存时的访存指令（比如访问页表）的次数，从而提高性能。

减少页表项数量：大页面减少了页表的层级和页表项的数量，减少了对页表的访问和管理，有助于降低内存访问的开销。

更少的页表开销：大页面需要的页表项较少，节省了内存和操作系统管理页表的开销。

劣势：

内部碎片：大页面可能导致内部碎片，因为如果一页内存没有完全使用，剩余的部分也会被占用，无法被其他进程利用。这种情况可能浪费一些内存空间。

不利于内存利用：大页面可能导致不连续的内存分配问题，因为它要求要分配的内存大小必须与页面大小相匹配。这可能会导致一些内存浪费和碎片化。

更大的换入/换出代价：大页面会增加换入/换出的代价。当需要换出一个页面时，需要换出更多的数据，这可能会增加I/O的开销。

TLB失效可能影响更多的数据：如果一个大页面的部分数据失效，可能会导致整个页面从TLB中移除，而不仅仅是部分失效的数据，这可能会增加内存访问的开销。

选择使用大页面还是分级页表，需要根据具体的应用场景和系统需求进行权衡。在一些特定情况下，使用大页面能够提供更好的性能，但在其他情况下可能会导致一些内存管理方面的问题。

实现不考虑实现开销和效率的LRU页替换算法（需要编程）challenge部分不是必做部分，不过在正确最后会酌情加分。需写出有详细的设计、分析和测试的实验报告。完成出色的可获得适当加分。

make qemu输出的解释：

特殊的内核符号 (Special kernel symbols) :

entry, etext, edata, end: 这些是特殊的内核符号, 指示内核的某些位置, 如代码段的起始 (entry)、代码段的结束 (etext)、数据段的结束 (edata)、内存结束 (end)。

Kernel executable memory footprint: 显示内核可执行内存的占用空间为70KB。

memory management: 使用了一个名为default_pmm_manager的内存管理器。

membegin, memend, mem_size: 显示了内存范围和总大小的信息。

物理内存映射 (Physical memory map) :

显示了内存的物理地址空间和内存范围。

check_alloc_page(): 该功能用于检查分配页的情况, 并表示已成功。

check_pgdir()、check_boot_pgdir()、check_vma_struct()、check_pgfault()、check_vmm(): 这些都是内核中的各种检查功能或测试功能, 检查内存管理、页目录等功能的正确性。

SWAP相关:

提到了Swap的一些管理信息, 包括了页的换入换出过程, 以及硬盘上的Swap分区中存储的页的管理情况。

page fault相关:

指出了发生的页面错误 (page fault) 情况, 包括了页面错误的地址、读写属性等信息。产生了一系列页面错误, 后续进行了swap in和swap out操作。

timer interrupts: 提到设置了定时器中断。

这些信息是针对内存管理、页面错误、swap管理等的测试和输出, 用于检查内核和内存管理功能的正确性。

当程序运行中访问内存产生page fault异常时, 如何判定这个引起异常的虚拟地址内存访问是越界、写只读页的“非法地址”访问还是由于数据被临时换出到磁盘上或还没有分配内存的“合法地址”访问?

判断内存访问异常的类型, 涉及到异常处理和错误代码。在操作系统中, 当发生页面错误 (Page Fault) 时, 异常处理程序会根据 CPU 提供的错误码 (Page Fault Error Code) 来识别异常的类型。这个错误码中包含了引起异常的具体原因。在RISC-V架构中, 有关页面错误的错误码通常包括以下信息:

读/写位: 指示了异常是由于读取还是写入操作引起的。当这个位为0时, 表示访问是一个读取操作; 为1时, 表示访问是一个写入操作。

存在位 (Present Bit): 在页面表项中指示了所请求的页面是否已经被载入内存。如果这个位是0, 表示所请求的页面不在内存中, 此时可能会发生页面错误。

保护位: 表明了所请求页面的保护属性 (例如, 只读页面等)。

地址位: 如果某些情况下发生地址错误, 错误码中可能包含引发异常的地址信息。

通过解析这些错误码的特定位, 可以区分出页面错误的不同类型。如果访问是越界的或者访问了只读页面, 通常在错误码中的位会指示这个情况。如果访问是合法的, 但是由于数据被临时换出到磁盘上或还没有分配内存, 通常异常处理程序会触发页面置换 (Swap) 或者分配页面的过程, 然后尝试重新执行产生异常的指令。

在异常处理程序中, 系统会根据错误码对访问类型进行识别, 并执行相应的操作, 例如从磁盘中加载数据 (如果是页面置换的情况), 或者分配内存页 (如果是未分配内存的合法地址访问) 等。

当程序运行时产生页面错误（page fault）异常时，操作系统会根据异常的类型进行判断：

访问越界或写只读页的非法地址：

如果虚拟地址访问位于用户空间但超出了合法范围，或者在用户空间访问的是只读页，通常这会被判定为越界或非法访问，触发了一个非法访问异常，操作系统会终止进程或者向用户发送一个相应的异常通知。

数据被临时换出到磁盘或未分配内存的合法地址访问：

如果页面错误是由于数据暂时被换出到磁盘（swap）或者是因为页面还没有被分配而导致的，这会被操作系统识别为一个“合法地址访问”，尽管发生了页面错误，但操作系统会尝试从磁盘中将数据换入内存，或者为这个访问分配合适的页面，然后继续进程的执行。

操作系统通过检查页表、页目录和其他内存管理数据结构来判断页面错误的具体原因，并采取相应的措施来解决或报告这个异常。

ucore在显示特殊内核符号地址后，展示了物理内存的映射信息，并进行了内存管理相关的检查。接着ucore进入页面置换和缺页异常处理的测试，并成功完成了页面置换测试。最后，ucore设置了定时器中断，在分页模式下响应时钟中断

何时进行请求调页/页换入换出处理？

请求调页（Page Fault）是指在访问某个虚拟地址的时候，相关页表条目没有包含对应的物理页，导致需要将相关的数据或指令从磁盘或其他辅助存储器加载到物理内存中，这时便会引发缺页异常。发生缺页异常时，需要进行请求调页处理，即进行页换入（Page In）和页换出（Page Out）。

页换入：当程序访问的虚拟页不在物理内存中时，需要将该虚拟页从磁盘加载到内存中的一个空闲物理页中。这个过程通常包括从磁盘读取页面内容，并将其复制到内存中的某个页框。

页换出：当物理内存不足以容纳所有需要被保留的页面时，需要为新进入的页腾出空间。这就需要将内存中的某些页（通常是很少使用的页面）移出到磁盘上，释放物理内存。这个过程通常包括将内存中页面的内容写回磁盘，然后更新页表，标记这个页面不再在内存中。

请求调页的处理发生在发生缺页异常时，操作系统需要根据缺页异常的地址和上下文信息来决定是从磁盘中读取页面还是写入页面内容。请求调页的目的是保证所需的页能够被正确加载到物理内存中，使程序能够继续运行。